

STANLEY COLLEGE OF ENGINEERING AND TECHNOLOGY FOR WOMEN
Chapel Road, Hyderabad
(Autonomus)



2025-2026

VI Semester BE

LABORATORY MANUAL
Of

COMPILER DESIGN LAB

(SPC0 603CS)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Prepared by

Mrs.B.Gnana Prasuna	Assistant Professor Dept. of CSE
----------------------------	---

GUIDELINES TO STUDENTS

1. Students should report at their respective lab as per the time table schedule with solved experiment in observation note book.
2. After completion of the experiment, student should get corrected the observation book by the staff in charge in that lab.
3. The record of the observations along with the detailed experimental procedure (algorithm, flowchart and program) of the experiment performed in the immediate last session should be submitted.
4. The batch wise division made in the beginning should be adhered to, and no mix up of student among different groups will be permitted later.
5. When the experiment is completed, students should shut down the computer.
6. Any damage of hardware or software or infrastructure due to negligence of student, accordingly the penalty will be awarded.
7. Students should prepare thoroughly for the concept, and solve the algorithm to perform the experiment before coming to Laboratory.

Dos and Don'ts:

- Don't install or download software or hardware or any equipment, materials, or processes that you are unfamiliar with get the proper training first.
- Perform only those experiments authorized by your teacher. Carefully follow all instructions, both written and oral.



STANLEY COLLEGE OF ENGINEERING & TECHNOLOGY FOR WOMEN

(Autonomous)

Abids, Hyderabad - 500 001, Telangana

Department of Computer Science and Engineering

Vision of the Institute

Empowering girl students through professional education integrated with values and character to make an impact in the world.

Mission of the Institute

- Providing quality engineering education for girl students to make them competent and confident to succeed in professional practice and advanced learning.
- Establish state-of-art-facilities and resources to facilitate world class education.
- Integrating qualities like humanity, social values, ethics, leadership in order to encourage contribution to society.

Vision of the Department

Empowering girl students with the contemporary knowledge in computer science engineering for their success in life.

Mission of the Department

- To impart quality education for girl students to learn and practice various hardware and software platforms prevalent in industry.
- To achieve self-sustainability and overall development through Research and Development activities.
- To provide education for life by focusing on the inculcation of human & moral values through an honest and scientific approach.
- To groom students with good attitude, team work and personality skills.

Program Educational Objectives (PEOs)

PEO1. Graduates shall have enhanced skills and contemporary knowledge in software and hardware technologies for professional excellence, towards successful employment, advanced learning and research.

PEO2. Graduates shall have life-long learning attitude, innovation and creativity to master latest technologies, devise solutions for realistic and social issues in the society.

PEO3. Graduates shall have good attitude and personality skills, ethical values, teamwork and leadership skill towards professionalism and ethical practices within the organization and the society.



STANLEY COLLEGE OF ENGINEERING & TECHNOLOGY FOR WOMEN

(Autonomous)

Abids, Hyderabad - 500 001, Telangana

Department of Computer Science and Engineering

Knowledge and Attitude Profile (WK)

WK1.	A systematic, theory-based understanding of the natural sciences applicable to the discipline and awareness of relevant social sciences.
WK2.	Conceptually-based mathematics, numerical analysis, data analysis, statistics and formal aspects of computer and information science to support detailed analysis and modelling applicable to the discipline.
WK3.	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.
WK4.	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas in the engineering discipline; much is at the forefront of the discipline.
WK5.	Knowledge, including efficient resource use, environmental impacts, whole-life cost, reuse of resources, net zero carbon, and similar concepts, that supports engineering design and operations in a practice area.
WK6.	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.
WK7.	Knowledge of the role of engineering in society and identified issues in engineering practice in the discipline, such as the professional responsibility of an engineer to public safety and sustainable development.
WK8.	Engagement with selected knowledge in the current research literature of the discipline, awareness of the power of critical thinking and creative approaches to evaluate emerging issues.
WK9.	Ethics, inclusive behavior and conduct. Knowledge of professional ethics, responsibilities, and norms of engineering practice. Awareness of the need for diversity by reason of ethnicity, gender, age, physical ability etc. with mutual understanding and respect, and of inclusive attitudes.



STANLEY COLLEGE OF ENGINEERING & TECHNOLOGY FOR WOMEN

(Autonomous)

Abids, Hyderabad - 500 001, Telangana

Department of Computer Science and Engineering

Program Outcomes (POs)

PO1.	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2.	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3.	Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4.	Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5.	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6.	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7.	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8.	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9.	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10.	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.
PO11.	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change. (WK8)

Program Specific Outcomes (PSOs)

PSO1.	Problem-Solving Skills: The ability to apply standard practices and strategies in software project development using open-ended programming environments to deliver a quality product for the benefit of students.
PSO2.	Design, Implement, Test and Evaluate a computer system, component or algorithm to meet desired needs and to solve a computational problem.

Course Code	Course Title					Core/Elective	
PC 613CS	COMPILER DESIGN LAB					Core	
Prerequisite	Contact Hours per Week				CIE	SEE	Credits
	L	T	D	P			
C Language	-	-	-	2	40	60	2

Course Objectives:

- To Learn usage of tools like LEX, YACC.
- To develop a code generator.
- To implement different code optimization schemes.

- Course Outcomes:
- After completing this course, the student will be able to:
- Generate scanner and parser from formal specification.
- Generate top down and bottom up tables using predictive parsing, SLR and LR parsing techniques.
- Apply the knowledge of YACC to syntax directed translation for generating intermediate code– 3 address code.
- Built a code generator using different intermediate codes and optimize the target code.

List of Experiments:

S. No.	Description of the program
1.	Program to recognize Tokens for a given simple C program.
2.	Program to implement Scanner Generation for a sample c program using LEX
3.	Program to implement LL(1) parser.
4.	Programs to implement LR parsers:- Shift Reduce parser SLR LALR parsers.
5.	Program to construct SDT for a simple expression.
6.	Program to implement Intermediate Code Generation.
7.	Program to implement Target Code Generation for a given Intermediate Code
8.	Program to implement Demonstration of constant propagation and common sub expression elimination.

Text Books:

1. Alfred V. Aho, Monica S. Lam, Ravi Sethi, & Jeffrey D. Ullman , Compilers :Principles, Techniques and Tools, 2nd Edition, Pearson Education, 2023.
2. Allen I.Holub, “Compiler Design in C”,Prentice Hall of India, 2003
3. Bennet J.P.,Introduction to Compiler Techniques, Tata McGraw-Hill, 2nd Edition.2003

Reference Books:

1. Kenneth C. Loudon, Compiler Construction: Principles and Practice, Thomson Learning Inc., 1997
2. P.Trembley and P.S.Sorenson, The Theory and Practice of Compiler Writing, TMH1985

List of Experiments mapping with POs, PSOs AND SDG

S. No.	Description of the program	POS	PSO	SDG
1.	Program to recognize Tokens for a given simple C program.	PO1, PO2, PO3, PO5, PO10, PO12	PSO1, PSO2	SDG 4, SDG 9
2.	Program to implement Scanner Generation for a sample c program using LEX	PO1, PO2, PO3, PO5, PO10, PO12	PSO1, PSO2	SDG 4, SDG 9
3.	Program to implement LL(1) parser.	PO1, PO2, PO3, PO4, PO5, PO10, PO12	PSO1, PSO2	SDG 4, SDG 9
4.	Programs to implement LR parsers:- Shift Reduce parser SLR LALR parsers.	PO1, PO2, PO3, PO4, PO5, PO10, PO12	PSO1, PSO2	SDG 4, SDG 9
5.	Program to construct SDT for a simple expression.	PO1, PO2, PO3, PO4, PO5, PO10, PO12	PSO1, PSO2	SDG 4, SDG 9
6.	Program to implement Intermediate Code Generation.	PO1, PO2, PO3, PO4, PO5, PO10, PO12	PSO1, PSO2	SDG 4, SDG 9
7.	Program to implement Target Code Generation for a given Intermediate Code	PO1, PO2, PO3, PO4, PO5, PO10, PO12	PSO1, PSO2	SDG 4, SDG 9, SDG 12
8.	Program to implement Demonstration of constant propagation and common sub expression elimination.	PO1, PO2, PO3, PO5, PO10, PO12	PSO1, PSO2	SDG 4, SDG 9

TRANSLATOR

A translator is a program that takes as input a program written in one language and produces as output a program in another language. Beside program translation, the translator performs another very important role, the error-detection. Any violation of the HLL specification would be detected and reported to the programmers. Important role of translator are:

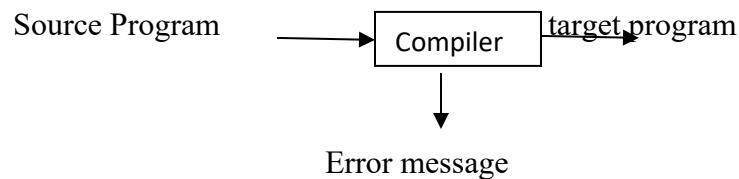
- 1, translating the HLL program input into an equivalent ML program.
- 2, providing diagnostic messages wherever the programmer violates specification of the HLL.

TYPE OF TRANSLATORS:-

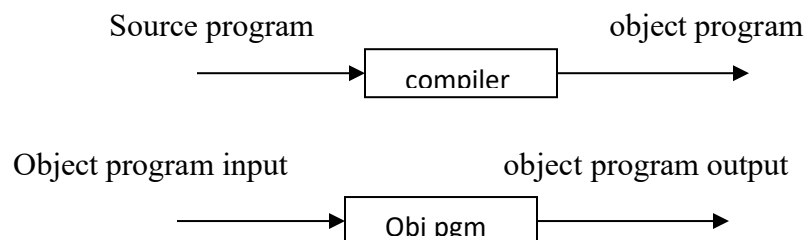
- INTERPRETOR
- COMPILER
- PREPROSESSOR

COMPILER :-

Compiler is a translator program that translates a program written in (HLL) the source program and translate it into an equivalent program in (MLL) the target program. As an important part of a compiler is error showing to the programmer.

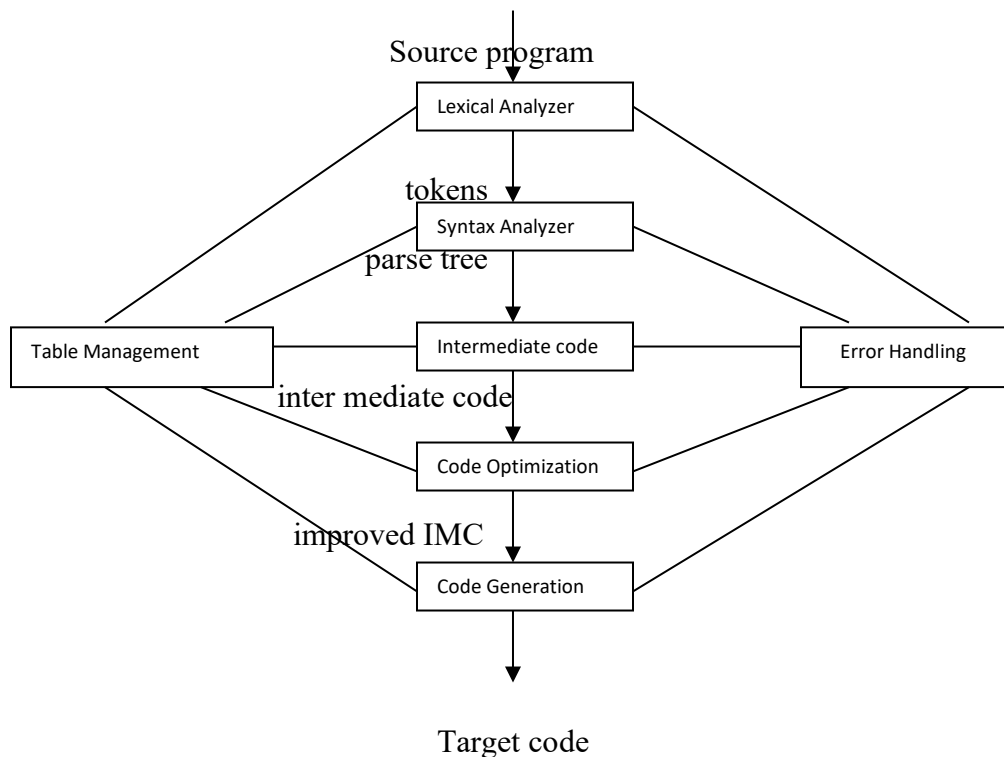


Executing a program written in HLL programming language is basically of two parts. the source program must first be compiled translated into a object program. Then the results object program is loaded into a memory executed.



STRUCTURE OF THE COMPILER DESIGN :-

Compilation process is partitioned into no-of-sub processes called '**phases**'.



Lexical Analysis:-

LA or Scanners reads the source program one character at a time, carving the source program into a sequence of atomic units called **tokens**.

Syntax Analysis:-

The second stage of translation is called Syntax analysis or parsing. In this phase expressions, statements, declarations etc... are identified by using the results of lexical analysis. Syntax analysis is aided by using techniques based on formal grammar of the programming language.

Intermediate Code Generations:-

An intermediate representation of the final machine language code is produced. This phase bridges the analysis and synthesis phases of translation.

Code Optimization :-

This is optional phase described to improve the intermediate code so that the output runs faster and takes less space.

Code Generation:-

The last phase of translation is code generation. A number of optimizations to **reduce the length of machine language program** are carried out during this phase. The output of the code generator is the machine language program of the specified computer.

Table Management (or) Book-keeping:-

This is the portion to **keep the names** used by the program and records essential information about each. The data structure used to record this information called a 'Symbol Table'.

Error Handlers:-

It is invoked when a flaw error in the source program is detected.

Passes:-

One (or) more phases one combined into a module called a pass.

Single-pass Compiler:-

All the phases are combined into a single

Multi-pass Compiler:-

More than one phase are combined into a number of groups called multi-pass.

(Multi pass compiler can be made to use less space than single pass compiler.).

LEXICAL ANALYSER:

Lexical analyser is the first phase of compiler. Which takes the source program as input and produces the stream of tokens as output.

Week1

1.Scanner program using C

Viva Questions

1. What is the purpose of lexical analysis?
2. Explain the phases of a compiler?
3. Define token?
4. What is the purpose of ungetc() function?
5. Define symbol table?

Program: 1

Aim: Program to recognize Tokens for a given simple C program

Algorithm for lexical analyzer:

function *lexan*: integer ; [Returns an integer encoding of token]

var *lexbuf*: array[0 .. 100] of char ;

c : char ;

begin

loop begin

read a character into *c* ;

if *c* is a blank or a tab **then**

do nothing

else if *c* is a newline **then**

lineno := *lineno* + 1

else if *c* is a digit **then begin**

set *tokenval* to the value of this and following digits ;

return NUM

end

else if *c* is a letter **then begin**

place *c* and successive letters and digits into *lexbuf* ;

p := lookup (*lexbuf*) ;

if *p* = 0 **then**

p := insert (*lexbf*, ID) ;

tokenval := *p*

return the *token* field of table entry *p*

end

else

set *tokenval* to NONE ; / * there is no attribute * /

return integer encoding of character *c*

end

end

Program:

1. Program to recognize Tokens for a given simple C program

*/*STANDALONE lexical analyzer PROGRAM*/*

```
#include<stdio.h>
#include<ctype.h>
#include<string.h>
int main()
{
FILE *input,*output;
int l=1;
int t=0;
int j=0;
int i,flag;
char ch,str[20];
char keyword[30][30]={"int","main","if","else","do","while","struct","goto"};
input=fopen("input.txt","r");
output=fopen("output.txt","w");
fprintf(output,"Line no. \t Token no.\t Token\t Lexeme\n\n");
while(!feof(input))
{
i=0;
flag=0;
ch=fgetc(input);
if( ch=='+' || ch=='-' || ch=='*' || ch=='/' || ch=='<' || ch=='>')
{
fprintf(output,"%6d\t\t %6d\t\t Operator\t %6c\n",l,t,ch);
t++;
}
else if( ch==';' || ch=='{' || ch=='}' || ch=='(' || ch==')' || ch=='?' || ch=='@' || ch=='!' || ch=='%')
{
fprintf(output,"%6d\t\t %6d\t\t Special symbol\t %6c\n",l,t,ch);
t++;
}
else if(isdigit(ch))
```

```

{
fprintf(output, "%d\t\t %d\t\t Digit\t\t %c\n", l, t, ch);
t++;
}
else if(isalpha(ch))
{
str[i]=ch;
i++;
ch=fgetc(input);
while(isalnum(ch)&& ch!=' ')
{
str[i]=ch;
i++;
ch=fgetc(input);
}
str[i]='\0';
for(j=0;j<=30;j++)
{
if(strcmp(str,keyword[j])==0)
{
flag=1;
break;
}
}
if(flag==1)
{
fprintf(output, "%d\t\t %d\t\t Keyword\t %s\n", l, t, str);
t++;
}
else
{
fprintf(output, "%d\t\t %d\t\t Identifier\t %s\n", l, t, str);
t++;
}
}
else if(ch=='\n')
{
l++;
}
}
fclose(input);
fclose(output);
output=fopen("output.txt", "r");
while(!feof(output))
{

```

```

        ch=fgetc(output);
        printf("%c",ch);
    }
    fclose(output);
    return 0;
}

```

INPUT.TXT

```

#include < stdio.h >
void main ()
{
int a,b,c ;
c = a + 9 ;
}

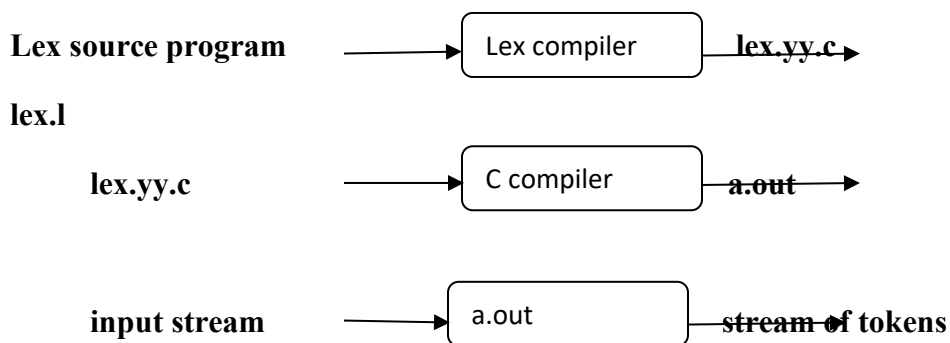
```

OUTPUT.TXT

Line no.	Token no.	Token	Lexeme
1	0	Identifier	include
1	1	Operator	<
1	2	Identifier	stdio
1	3	Identifier	h
1	4	Operator	>
2	5	Identifier	void
2	6	Keyword	main
2	7	Special symbol	(
2	8	Special symbol	)
3	9	Special symbol	{
4	10	Keyword	int
4	11	Identifier	a
4	12	Identifier	b
4	13	Identifier	c
4	14	Special symbol	;
5	15	Identifier	c
5	16	Special symbol	=
5	17	Identifier	a
5	18	Operator	+
5	19	Digit	9
5	20	Special symbol	;
6	21	Special symbol	}

2. Program to implement Scanner Generation for a sample c program using LEX

LEX: LEX is one of the compiler construction tools. Through this we can generate stream of tokens from the given input stream. Compilation process of it is shown bellow.



Structure of lex specification File:

It contains three parts

%{

Declaration part

%}

%%

Rule section -----> patterns { Action }

Auxalary functions

First, a specification of a lexical analyzer is prepared by creating a program lex.l in the Lex language. Then, lex.l is run through the Lex compiler to produce a C program lex. yy. c. The program lex. yy. c consists of a tabular representation of a transition diagram constructed from the regular expressions of

lex. 1, together with a standard routine that uses the table to recognize lexemes. The actions associated with regular expressions in lex.l are pieces of C code and are carried over directly to lex.yy. c. Finally, lex.yy. c is run through the C compiler to produce an object program a.oat, which is the lexical analyzer that transforms an input stream into a sequence of tokens.

Viva Questions

- 1) What is LEX lexical analyzer generator?
- 2) Differentiate token,lexeme,pattern?
- 3) What rule section contains ?
- 4) What Declaration section contains?
- 5) What Auxiliary procedure section contains?

2. Program to implement Scanner Generation for a sample c program using LEX

Procedure:

1. Start the program
2. Open a file file.c in read and include the yylex() tool for input scanning.
3. Define the alphabets and numbers.
4. Print the preprocessor, function, keyword using yytext.lex tool.
6. Print the relational, assignment and all the operator using yytext() tool.
7. Also scan and print where the loop ends and begins.
8. Use yywrap() to enter an error.
9. Stop the program.

Program

/*program name is lexp.l*/

%{

/*program to recognize a c program*/

int COMMENT=0;

%}

identifier [a-zA-Z][a-zA-Z0-9]*

%%

#. * {printf ("\n%s is PREPROCESSOR DIRECTIVE",yytext);}

int |

float |

char |

double |

while |

for |

do |

if |

break |

continue |

void |

switch |

case |

long |

```

    struct |
    const |
    typedef |
    return |
    else |

goto {printf("\n\t%s is KEYWORD",yytext);}

"*/"      {COMMET=1;}

/*{printf("\n\n\t%s is a COMMENT\n",yytext);}*/

{identifier}\(  {if(!COMMENT)printf("\n\nFUNCTION\n\t%s",yytext);}

\{           {if(!COMMENT) printf("\n BLOCK BEGINS");}

\}           {IF(!COMMENT) printf("\n BLOCK ENDS");}

{identifier}(\[[0-9]*\])? {if(!COMMENT)

printf("\n%s IDENTIFIER", yytext);

}

\".*\"      {if(!COMMENT) printf("\n\t%s is a STRING",yytext);}

[0-9]+      {if(!COMMENT) printf("\n\t%s IS A NUMBER",YYTEXT);}

\)(;)?      {if(!COMMENT) printf("\n\t");ECHO;printf("\n");}

\((        ECHO;

=           {if(!COMMENT)printf("\n\t%s is an ASSIGNMENT OPERATOR",yytext);}

\<= |

\>= |

\< |

== |

\>         {if(!COMMENT) printf("\n\t%s is a RELATIONAL
OPERATOR",yytext);

```

```

}

|\n    ;

%%

int main(int argc,char **argv) /* MAIN FUNCTION */
{
    if(argc > 1)
    {
        FILE *file;
        File = fopen(argv[1],”r”);
        if(!file)
        {
            printf(“could not open %s\n”,argv[1]);
            exit(0);
        }
        yyin=file;
    }
    yylex();
    printf(“\n\n”);
    return 0;
}

int yywrap()
{

```

```
        return 0;

    }

/*-----End of lexp.l-----*/

/*Program named as area.c*/

#include<stdio.h>
#include<stdlib.h>

double area_of_circle(double r);

int main()
{
    if(argc<2) /*this is a single line comment*/
    {
        printf("usage: %s radius\n",argv[0]);
        exit(1);
    }
    else
    {

/*this is a double line comment */

        double radius=atof(argv[1]);
        double area=area_of_circle(radius);
```

```

        printf("Area of circle with radius %f=%f\n",radius,area);
    }
    return 0;    /*returning 0 */
}

```

OUTPUT:

[localhost@gnanaprasuna]#lex lexp.l

[localhost@gnanaprasuna]#cc lex.yy.c

[localhost@gnanaprasuna]#./a.out area.c

#include<stdio.h> is a PREPROCESSOR DIRECTIVE

#include<stdlib.h> is a PREPROCESSOR DIRECTIVE

double is a KEYWORD

Function

```
area_of_circle(
```

double is KEYWORD

r is an IDENTIFIER

```
);
```

int is a KEYWORD

Function

```
main
(
    int is a KEYWORD
    argc IDENTIFIER
    char is a KEYWORD
    argv[] IDENTIFIER
)
```

BLOCK BEGINS

```
if is a KEYWORD
(
    argc IDENTIFIER
    < is a RELATIONAL OPERATOR
    2 is a NUMBER
)
```

```
/*This is a single line comment*/ is a COMMENT
```

BLOCK BEGINS

FUNCTION

```
printf(
    "Usage: %s radius \n" is a STRING
```



```
argv[0] IDENTIFIER  
);
```

FUNCTION

```
exit(  
1 is a NUMBER  
);
```

BLOCK ENDS

else is Akeyword

BLOCK BEGINS

```
double is KEYWORD  
radius IDENTIFIER  
= is an ASSIGNMENT OPERATOR
```

FUNCTION

```
atof(  
argv[1] IDENTIFIER  
  
);  
  
double is a KEYWORD  
area IDENTIFIER
```

= is an ASSIGNMENT OPERATOR

FUNCTION

printf(

“Area of circle with radius %f=%f\n” is a STRING

radius IDENTIFIER

area IDENTIFIER

);

BLOCK ENDS

return is a KEYWORD

0 is a NUMBER

//Returning 0 is a COMMENT

BLOCK ENDS

3. Program to implement LL(1) parser.

LL(1) Parser:

Algorithm:

Input: A CFG

Output: Is the predictive parsing table

Procedure:

1. For each production in the grammar G the following operations are done
 - a. For each terminal in the $FIRST(\alpha)$, add $A \rightarrow \alpha$ to $M[A, a]$.
 - b. manipulate ϵ
 - If $\epsilon \in FIRST(\alpha)$, then for each terminal $b \in FOLLOW(A)$ add $A \rightarrow \alpha$ to $M[A, b]$.
 - If $\epsilon \in FIRST(\alpha)$, and $\$$ in $FOLLOW(A)$ then add $A \rightarrow \alpha$ to $M[A, \$]$.
2. An error can be detected from empty entries in table $M[A, a]$, since there is no production for that entry with a pair A and a. For LL(1) grammars there is one production for each entry. For other grammars there can be several productions in one entry.

Implementation of LL(1) parsing table

```
#include<stdlib.h>
#include<stdio.h>
#include<string.h>
#include<conio.h>
#include<ctype.h>

void main()
{
    int i,j,h,k,r,m,n,t,nt;
    char prod[15][10], first[15][30], follow[15][30], terminals[10], nonterminals[10];
    static char table[10][10][10]={" "};

    *table[0][0]='X';
```

```

//clrscr();
printf("total no of productions\t no of terminals and \t nonterminals");
scanf("%d%d%d",&n,&t,&nt);
printf("\n\nenter production in this format LEFT-->RIGHT\n");
for(i=0;i<n;i++)
{
    printf("\nenter production%d \n", i+1);
    scanf("%s",&prod[i]);
}
printf("\nenter the values of non terminals\n");

    scanf("%s",nonterminals);

printf("\nenter the values of terminals\n");

    scanf("%s",terminals);

printf("\nenter the first values\n");
for(i=0;i<n;i++)
{
    scanf("%s",&first[i]);
}
printf("\nenter the follow values\n");
for(i=0;i<n;i++)
{
    scanf("%s",&follow[i]);
}
for(i=1,j=1;i<=nt,j<=t;i++,j++)
{

    *table[i][0]=nonterminals[i-1];
    *table[0][j]=terminals[j-1];

}
h=t;
*table[0][h+1]='$';
for(i=0;i<=n;i++)
{
    //check=prod[i][0];
    for(j=1;j<=nt;j++)
    {
        if(prod[i][0]==(*table[j][0]))
            r=j;
    }
    for(k=0;k<=strlen(first[i]);k++)
    {

```

```

        if(prod[i][4]!='#')
        {
            for(j=1;j<=t+1;j++)
            {
                if(isupper(prod[i][4]))
                {
                    if(first[i][k]==*table[0][j])
                    strcpy(table[r][j],prod[i]);
                }
                else
                {
                    if(first[i][k]==*table[0][j] && prod[i][4]==first[i][k])
                    strcpy(table[r][j],prod[i]);
                }
            }
        }
        else
        {
            for(m=0;m<=strlen(follow[i]);m++)
            {
                for(j=1;j<=t+1;j++)
                { if(follow[i][m]==*table[0][j])
                    strcpy(table[r][j],prod[i]);
                }
            }
        }
    }

    printf("-----");
    for( i=0;i<=nt;i++)
    {
        printf("\n\n");
        for(j=0;j<=t+1;j++)
        {
            printf("%s\t",table[i][j]);

        }
    }
}

```

```
printf("\n-----");
}
```

Output:

```
C:\Windows\system32\cmd.exe
total no of productions no of terminals and nonterminals 3 3 3
enter production in this format LEFT-->RIGHT
enter production1
E-->A+B
enter production2
A-->a
enter production3
B-->b
enter the values of non terminals
EAB
enter the values of terminals
*ab
enter the first values
a
a
b
enter the first values
$
+
$
-----
X      +      a      b      $
E      E-->A+B
A      A-->a
B      B-->b
-----Press any key to conti
nue . . .
```

4. Programs to implement LR parsers:-

- Shift Reduce parser
- SLR
- LALR parsers.

Viva Questions

1. Explain Back Tracking?
2. Why we call syntax analyzer as parser?
3. What are the problems in Top Down Parser?

4. Define CFG?

5. Define ambiguity?

a. Shift Reduce parser

```
#include <stdio.h>
#include <string.h>

struct ProductionRule
{
    char left[10];
    char right[10];
};

int main()
{
    char input[20], stack[50], temp[50], ch[2], *token1, *token2, *substring;
    int i, j, stack_length, substring_length, stack_top, rule_count = 0;
    struct ProductionRule rules[10];

    stack[0] = '\0';

    // User input for the number of production rules
    printf("\nEnter the number of production rules: ");
    scanf("%d", &rule_count);

    // User input for each production rule in the form 'left->right'
    printf("\nEnter the production rules (in the form 'left->right'): \n");
    for (i = 0; i < rule_count; i++)
    {
        scanf("%s", temp);
        token1 = strtok(temp, "->");
        token2 = strtok(NULL, "->");
        strcpy(rules[i].left, token1);
        strcpy(rules[i].right, token2);
    }

    // User input for the input string
    printf("\nEnter the input string: ");
    scanf("%s", input);
```

```

i = 0;
while (1)
{
    // If there are more characters in the input string, add the next character to the stack
    if (i < strlen(input))
    {
        ch[0] = input[i];
        ch[1] = '\0';
        i++;
        strcat(stack, ch);
        printf("%s\t", stack);
        for (int k = i; k < strlen(input); k++)
        {
            printf("%c", input[k]);
        }
        printf("\tShift %s\n", ch);
    }

    // Iterate through the production rules
    for (j = 0; j < rule_count; j++)
    {
        // Check if the right-hand side of the production rule matches a substring in the stack
        substring = strstr(stack, rules[j].right);
        if (substring != NULL)
        {
            // Replace the matched substring with the left-hand side of the production rule
            stack_length = strlen(stack);
            substring_length = strlen(substring);
            stack_top = stack_length - substring_length;
            stack[stack_top] = '\0';
            strcat(stack, rules[j].left);
            printf("%s\t", stack);
            for (int k = i; k < strlen(input); k++)
            {
                printf("%c", input[k]);
            }
            printf("\tReduce %s->%s\n", rules[j].left, rules[j].right);
            j = -1; // Restart the loop to ensure immediate reduction of the newly derived
production rule

```



```
    }  
}  
  
// Check if the stack contains only the start symbol and if the entire input string has been  
processed  
if (strcmp(stack, rules[0].left) == 0 && i == strlen(input))  
{  
    printf("\nAccepted");  
    break;  
}  
  
// Check if the entire input string has been processed but the stack doesn't match the start  
symbol  
if (i == strlen(input))  
{  
    printf("\nNot Accepted");  
    break;  
}  
}  
  
return 0;  
}
```

OUTPUT:

```

Enter the number of production rules: 4

Enter the production rules (in the form 'left->right'):
E->E+E
E->E*i
E->(E)
E->i

Enter the input string: i*i+i
i      *i+i    Shift i
E      *i+i    Reduce E->i
E*     i+i     Shift *
E*i    +i      Shift i
E*iE   +i      Reduce E->i
E      +i      Reduce E->E*i
E+     i       Shift +
E+i          Shift i
E+E          Reduce E->i
E          Reduce E->E+E

Accepted

```

b. SLR parser

Input: input string to accept

Output: String Acceptance of the grammar

Procedure:

1. Construct $C = \{ I_0, I_1, \dots, I_n \}$, the collection of sets of LR(0) items for G' .
2. State i is constructed from I_i . The parsing actions for state i are determined as follow :
 - If $[A \rightarrow ?a?]$ is in I_i and $\text{GOTO}(I_i, a) = I_j$, then set $\text{ACTION}[i, a]$ to “shift j ”. Here a must be terminal.
 - If $[A \rightarrow ?.]$ is in I_i , then set $\text{ACTION}[i, a]$ to “reduce $A \rightarrow ?$ ” for all a in $\text{FOLLOW}(A)$; here A may not be S' .
 - Is $[S \rightarrow S.]$ is in I_i , then set $\text{action}[i, \$]$ to “accept”. If any conflicting actions are generated by the above rules we say that the grammar is not SLR.
3. The goto transitions for state i are constructed for all nonterminals A using the rule:
if $\text{GOTO}(I_i, A) = I_j$ then $\text{GOTO}[i, A] = j$.
4. All entries not defined by rules 2 and 3 are made error.

DESCRIPTION:

1. Declare parser table which contains the parser actions.
2. Initialize a input buffer for storing input string and a stack for storing the string to be parsed.
3. Check the entry in the parser table by taking the values of stack pointer and input buffer.
4. If the entry in the parser table is “shift” push the input string token into stack.
5. If the entry in the parser table is “reduce” perform the reduction of the corresponding production rule by doing pop and push operations on stack.
6. Repeat it until we reach the accept state which represents the acceptance of string.

Program:

```
#include<stdio.h>
#include<string.h>
#include<ctype.h>

/* SLR parser for the grammar
  E->E+T (1)
  E->T (2)
  T->T*F (3)
  T->F (4)
  F->(E) (5)
  F->ID (6)
*/

/* Stack structure and fuctions */
```

/* The only thing important here is that the value associated with each name of the macro should be unique. They have no relation to the states in the dfa */

```
#define S4 1
#define S5 2
#define S6 3
#define S7 4
#define R1 5
#define R2 6
#define R3 7
#define R4 8
#define R5 9
#define R6 10
#define S11 11
#define AC 11      /* ACCEPT */
#define ER -1      /* ERROR */
```

/* the parsing table */

```
int table[][9]= {
    {S5,ER,ER,S4,ER,ER,1,2,3},
    {ER,S6,ER,ER,ER,AC,ER,ER,ER},
    {ER,R2,S7,ER,R2,R2,ER,ER,ER},
    {ER,R4,R4,ER,R4,R4,ER,ER,ER},
    {S5,ER,ER,S4,ER,ER,8,2,3},
    {ER,R6,R6,ER,R6,R6,ER,ER,ER},
    {S5,ER,ER,S4,ER,ER,ER,9,3},
    {S5,ER,ER,S4,ER,ER,ER,ER,10},
    {ER,S6,ER,ER,S11,ER,ER,ER,ER},
    {ER,R1,S7,ER,R1,R1,ER,ER,ER},
    {ER,R3,R3,ER,R3,R3,ER,ER,ER},
    {ER,R5,R5,ER,R5,R5,ER,ER,ER}
};
```

```
#define STRING_SIZE 20
```

```
char string[STRING_SIZE];
```

```
int i=0;
```

```
int save;
```

```
#define STACK_SIZE 40
```

```

typedef struct {
    int list[STACK_SIZE];
    int top;
} Stack;

void initialize(Stack *s) {
    s->top=-1;
}
void push(int value,Stack *s) {
    s->list[++(s->top)]=value;
}
int pop(Stack *s) {
    return(s->list[(s->top)--]);
}
int isempty(Stack *s) {
    return(s->top==-1);
}
int peek(Stack *s) {
    return(s->list[s->top]);
}
int stacksize(Stack *s) {
    return((s->top)+1);
}

#define ID 0
#define ADD 1
#define MULT 2
#define OPBR 3
#define CLBR 4
#define DOLLAR 5
#define E 6
#define T 7
#define F 8

short int gettoken() {
    /* ignore blanks */
    while(string[i]==' ')
        i++;
    /* definition for identifier */

```

```

if(isalpha(string[i])) {
    save=i;
    i++;
    while(string[i]!=0 && string[i]!='*' && string[i]!='+' && string[i]!=')' && string[i]!='(') {
        if(!isalnum(string[i++]))
            error();
    }
    return ID;

}
else if(string[i]=='+') {
    save=i;
    i++;
    return ADD;
}
else if(string[i]=='*') {
    save=i;
    i++;
    return MULT;
}
else if(string[i]=='(') {
    save=i;
    i++;
    return OPBR;
}
else if(string[i]==')') {
    save=i;
    i++;
    return CLBR;
}
else if(string[i]==0) {
    return DOLLAR;
}
}
error() {
    printf("Bad Bad error.\n");
    exit(0);
}
int parse() {
    int token;

```

```

Stack stack;
int state;
int j;
int action;
int previous;
initialize(&stack);
push(0,&stack);
token=0;
while(1) {
    token=gettoken();
    action=table[peek(&stack)][token];
    switch(action) {
    case S4:
        push(token,&stack);
        push(4,&stack);
        break;
    case S5:
        push(token,&stack);
        push(5,&stack);
        break;
    case S6:
        push(token,&stack);
        push(6,&stack);
        break;
    case S7:
        push(token,&stack);
        push(7,&stack);
        break;
    case AC:
        return 1;
    case ER:
        error();

    }
    if(action>=5 && action <=10) {
        while(action>=5 && action <=10) {
            action=table[peek(&stack)][token];
            switch(action) {
            case R1:
                for(j=0;j<6;j++)

```

```
    pop(&stack);
    state=table[peek(&stack)][E];
    if(state!=ER) {
        push(E,&stack);
        push(state,&stack);
    }
    else
        error();
    break;
case R2:
    pop(&stack);
    pop(&stack);
    state=table[peek(&stack)][E];
    if(state!=ER) {
        push(E,&stack);
        push(state,&stack);
    }
    else
        error();
    break;
case R3:
    for(j=0;j<6;j++)
        pop(&stack);
    state=table[peek(&stack)][T];
    if(state!=ER) {
        push(T,&stack);
        push(state,&stack);
    }
    else
        error();
    break;
case R4:
    pop(&stack);
    pop(&stack);
    state=table[peek(&stack)][T];
    if(state!=ER) {
        push(T,&stack);
        push(state,&stack);
    }
    else
```



```

    error();
    break;
case R5:
    for(j=0;j<6;j++)
        pop(&stack);
    state=table[peek(&stack)][F];
    if(state!=ER) {
        push(F,&stack);
        push(state,&stack);
    }
    else
        error();
    break;
case R6:
    pop(&stack);
    pop(&stack);
    state=table[peek(&stack)][F];
    if(state!=ER) {
        push(F,&stack);
        push(state,&stack);
    }
    else
        error();
    break;
case AC:
    return 1;
case ER:
    error();
}
}
i=save;
}
}
return 0;
}
int main() {
    printf("Enter the string: ");
    scanf("%s",string);
    if(parse()) {
        printf("Success in parsing.\n");

```

```

}
else
    error();
}

```

Output:

```
[localhost@ gnanaprasuna]#cc slr.c
```

```
[localhost@ gnanaprasuna]# ./a.out
```

Enter the strinh to be par sed.Press Ctrl Z at the end

A+B

Success in parsing

Prasuna

Success in parsing

Ctrl Z

```
[localhost@gnanaprasuna]#
```

c.LALR

YACC tool:

YACC is one of the compiler construction Tools which will produce a parser for a given grammar. Yacc (Yet Another Compiler-Compiler) is a parser-generator for LALR-languages. The Yacc specification is compiled to a C program and the execution of that program acts as the parser. The specification contains the three parts:

declarations

%%

translation rules

%%

supporting C routines

The *declaration* part has two sections, the first one contains ordinary C declarations delimited by %{ and %}.

The second section contains declaration of grammar tokens, which can be used in the later parts. If Lex is used with Yacc then these tokens will be available also in Lex.

The translation rules part contains the productions of the language.
The syntax is basically

```
< head > : < body >1 {< semantic action >1}
          | < body >2 {< semantic action >2}
          ...
          | < body >n {< semantic action >n}
          ;
```

For the production

$$E \rightarrow E + T | T$$

the translational rules are

```
expr : expr '+' term    { $$ = $1 + $3; }
     | term
     ;
```

The semantic action $$$ = \1 ; is the default and thus, can be omitted.

MOTIVATION

The LALR (Look Ahead-LR) parsing method is between SLR and Canonical LR both in terms of power of parsing grammars and ease of implementation. This method is often used in practice because the tables obtained by it are considerably smaller than the Canonical LR tables, yet most common syntactic constructs of programming languages can be expressed conveniently by an LALR grammar. The same is almost true for SLR grammars, but there are a few constructs that can not be handled by SLR techniques

YACC PROGRAMS

program for the LALR parser for the following grammar

Procedure:

1. Create a lex specification file to recognize tokens in the grammar
2. Create a Yacc specification file to parse the tokens in the grammar
3. Compile lex and yacc files to generate C files
4. Now compile both the c files using C compiler to get executable file
5. Run the executable file to get the desired output by providing necessary input

/* lex specification file parser.*/

```
%{
#include<stdio.h>
```

```

#include "y.tab.h"
%}
%%
[0-9]+ {yyval.dval=atof(yytext);
return DIGIT;
}
\n|. return yytext[0];
%%

/* yacc specification file parser.y*/

%{
/*This YACC specification file generates the LALR parser for the program */
#include<stdio.h>
%}
%union
{
double dval;
}
%token <dval> DIGIT
%type <dval> expr
%type <dval> term
%type <dval> factor
%%
line: expr '\n'      {
printf("%g\n", $1);
}
;
expr: expr '+' term {$$=$1 + $3 ;}
    | term
    ;
term: term '*' factor {$$=$1 * $3 ;}
    | factor
    ;
factor: '(' expr ')' {$$=$2 ;}
    | DIGIT

;
%%
int main()
{
yyparse();

```

```

}
yyerror(char *s)
{
printf("%s",s);
}

```

Output:

```

[localhost@ gnanaprasuna]#lex LALAR.l
[localhost@ gnanaprasuna]# yacc -d LALR.y
[localhost@ gnanaprasuna]#cc lex.yy.c y.tab.c
[localhost@ gnanaprasuna]# ./a.out
2+3
5.0000
5*4
20

```

5. Program to construct SDT for a simple expression.

Program code:

```

#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 50
#define LINE 200

// trim leading/trailing spaces (in-place)
void trim(char *s) {
    int i = 0;
    while (isspace((unsigned char)s[i])) i++;
    if (i > 0) memmove(s, s + i, strlen(s + i) + 1);

    int n = (int)strlen(s);
    while (n > 0 && isspace((unsigned char)s[n - 1])) s[--n] = '\0';
}

int main() {
    int n;
    char prod[MAX][LINE];

    printf("Enter number of productions: ");
    if (scanf("%d", &n) != 1 || n <= 0 || n > MAX) return 0;

```

```

getchar(); // consume newline

printf("Enter productions (example: E -> E + T):\n");
for (int i = 0; i < n; i++) {
    fgets(prod[i], LINE, stdin);
    prod[i][strcspn(prod[i], "\n")] = '\0';
    trim(prod[i]);
}

printf("\nOutput with SDT actions:\n");
for (int i = 0; i < n; i++) {
    char line[LINE];
    strcpy(line, prod[i]);

    // We will detect RHS by splitting on "->"
    char *arrow = strstr(line, "->");
    if (!arrow) {
        // not a valid production, print as-is
        printf("%s\n", prod[i]);
        continue;
    }

    // LHS is before arrow, RHS after arrow
    char lhs[50], rhs[LINE];
    int lhs_len = (int)(arrow - line);
    strncpy(lhs, line, lhs_len);
    lhs[lhs_len] = '\0';
    strcpy(rhs, arrow + 2);

    trim(lhs);
    trim(rhs);

    // Print original production first
    printf("%s\n", prod[i]);

    // Decide which SDT action to append (based on RHS)
    // Case 1: E -> E + T => print('+')
    if (strchr(rhs, '+') != NULL) {
        printf("{ print('+') }\n");
    }
    // Case 2: T -> 0 => print('0')
    else if (strcmp(rhs, "0") == 0) {
        printf("{ print('0') }\n");
    }
    // Case 3: T -> 1 => print('1')
    else if (strcmp(rhs, "1") == 0) {

```

```

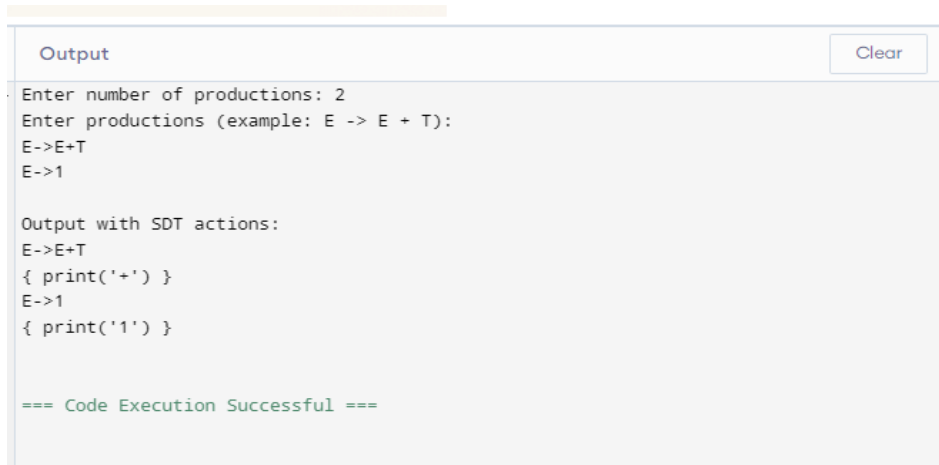
        printf("{ print('1') }\n");
    }

    // For rules like E -> T, no action is added (as per your example)
}

return 0;
}

```

Output:



```

Output
Clear

Enter number of productions: 2
Enter productions (example: E -> E + T):
E->E+T
E->1

Output with SDT actions:
E->E+T
{ print('+') }
E->1
{ print('1') }

=== Code Execution Successful ===

```

6. Intermediate code generation

Three-address code is a sequence of instructions of the form

$$x - y \text{ op } z$$

where x , y , and z are names, constants, or compiler-generated temporaries; and **op** stands for an operator.

Data structures for implementing three address code are as follows

- Quadruples
 - Has four fields: op, arg1, arg2 and result
- Triples
 - Temporaries are not used and instead references to instructions are made
- Indirect triples
 - In addition to triples we use a list of pointers to triples

Intermediate code generation**Procedure:**

1. Create a lex specification file to recognize tokens in the grammar
2. Create a Yacc specification file to parse the tokens in the grammar
3. Compile lex and yacc files to generate C files
4. Now compile both the c files using C compiler to get executable file
5. Run the executable file to get the desired output by providing necessary input

Program:

```
//Inter.l
```

```
%{
```

```
#include "y.tab.h"
```

```
extern char yyval;
```

```
%}
```

```
NUMBER [0-9]+
```

```
LETTER [a-zA-Z]+
```

```
%%
```

```
{NUMBER} {yyval.sym=(char)yytext[0];
```

```
    return NUMBER;}
```

```
{LETTER} {yyval.sym=(char)yytext[0];return LETTER;}
```

```
\n {return 0;}
```

```
. {return yytext[0];}
```

```
//inter.y
```



```

%{
#include<stdio.h>
#include<string.h>
int nIndex=0;
struct InterCode
{
char operand1;
char operand2;
char opera;
};
}%
%union
{
char sym;
}
%token <sym> LETTER NUMBER
%type <sym> expr
%left '-' '+'
%right '*' '/'
%%
statement:LETTER '=' expr ';' {AddToTable((char)$1,(char)$3,'=');}
        |expr ';'
        ;
expr : expr '+'expr {$$=AddToTable((char)$1,(char)$3,'+');}
     |expr '-'expr {$$=AddToTable((char)$1,(char)$3,'-');}

```

```

|expr '*'expr {$$=AddToTable((char)$1,(char)$3,'*');}

|expr '/'expr {$$=AddToTable((char)$1,(char)$3,'/');}

|('expr') {$$=(char)$2;}

|NUMBER {$$=(char)$1;}

|LETTER {$$=(char)$1;}

;

%%

yyerror(char *s)
{
printf("%s",s);
exit(0);
}

struct InterCode Code[20];

char AddToTable(char operand1,char operand2,char opera){
char temp='A';

Code[nIndex].operand1=operand1;
Code[nIndex].operand1=operand2;
Code[nIndex].opera=opera;

nIndex++;

temp++;

return temp;
}

ThreeAddressCode()
{
int nCnt=0;

```

```

char temp='A';

printf("\n\n\tTHREE ADDRESS CODE");

temp++;

while(nCnt<nIndex)
{
printf("%c:=\t",temp);

if(isalpha(Code[nCnt].operand1))

    printf("%c\t",Code[nCnt].operand1);

else

    printf("%c\t",temp);

printf("%c\t",Code[nCnt].opera);

if(isalpha(Code[nCnt].operand2))

printf("%c\t",Code[nCnt].operand2);

else

printf("%c\t",temp);

printf("\n");

nCnt++;

temp++;

}

}

void Quadruples()

{

int nCnt=0;

char temp='A';

temp++;

```

```

printf("\n\n\tQuadruples\n");

printf("\n ID OPERATOR OPERAND1 OPERAND2 RESULT\n");

while(nCnt<nIndex)
{
printf("\n(%d) \t %c\t",nCnt,Code[nCnt].opera);

if(isalpha(Code[nCnt].operand1))
printf("%c\t",Code[nCnt].operand1);
else
printf("%c\t",temp);

if(isalpha(Code[nCnt].operand2))
printf("%c\t",Code[nCnt].operand2);
else
printf("%c\t",temp);
printf("\n");
nCnt++;
temp++;
}
}

main()
{
printf("\nEnter the expression\n");

yyparse();

ThreeAddressCode();

Quadruples();

```

```

}
yywrap()
{
return 1;
}

```

Output:

```

administrator@ubuntu:~/Desktop$ flex th.l
administrator@ubuntu:~/Desktop$ yacc -d th.y
administrator@ubuntu:~/Desktop$ gcc lex.yy.c y.tab.c -ll -lm
administrator@ubuntu:~/Desktop$ ./a.out

```

```

administrator@ubuntu:~/Desktop$ ./a.out

```

Enter the Expression: $a=((b+c)*(d/e));$
 THREE ADDRESS CODE

```

B := b + c
C := d / e
D := B * C
E := a = D

```

QUADRAPLE CODE

```

0 + b c G
1 / d e H
2 * B C I
3 = a D J

```

7. Program to implement Target Code Generation for a given Intermediate Code

Algorithm:

Input: A text file which contain three address code in the form Triple form.

Output: Machine language

Procedure:

1. Read the text file which contain three address code in the form Triple form.

2. If the first character is '=' then output

LOAD operand1 to R0

3. If the first character is '+' then output

ADD Operand 1 Operand 2.

3. Similar procedure can be applied for all other operators.

4. If the first character is 'goto' then output

JGT 5,label#11

5. If the first character is 'print' then output

OUTPUT operand 1

```
#include<stdio.h>
```

```
#include<stdlib.h>
```

```
#include<string.h>
```

```
int label[20];
```

```
int no=0;
```

```
int main()
```

```
{
```

```
FILE *fp1,*fp2;
```

```
char fname[10],op[10],ch;
```

```
char operand1[8],operand2[8],result[8];
```

```
int i=0,j=0;
```

```
printf("\n Enter filename of the intermediate code");
scanf("%s",&fname);
fp1=fopen(fname,"r");
fp2=fopen("target.txt","w");
if(fp1==NULL || fp2==NULL)
{
printf("\n Error opening the file");
exit(0);
}
while(!feof(fp1))
{
fprintf(fp2,"\n");
fscanf(fp1,"%s",op);
i++;
if(check_label(i))
fprintf(fp2,"\nlabel#%d",i);
if(strcmp(op,"print")==0)
{
fscanf(fp1,"%s",result);
fprintf(fp2,"\n\t OUT %s",result);
}
if(strcmp(op,"goto")==0)
{
fscanf(fp1,"%s %s",operand1,operand2);
fprintf(fp2,"\n\t JMP %s,label#%s",operand1,operand2);
```

```

label[no++]=atoi(operand2);
}
if(strcmp(op,"[]")==0)
{
fscanf(fp1,"%s %s %s",operand1,operand2,result);
fprintf(fp2,"\n\t STORE %s[%s],%s",operand1,operand2,result);
}
if(strcmp(op,"uminus")==0)
{
fscanf(fp1,"%s %s",operand1,result);
fprintf(fp2,"\n\t LOAD -%s,R1",operand1);
fprintf(fp2,"\n\t STORE R1,%s",result);
}
switch(op[0])
{
case '*': fscanf(fp1,"%s %s %s",operand1,operand2,result);
fprintf(fp2,"\n\t LOAD",operand1);
fprintf(fp2,"\n\t LOAD %s,R1",operand2);
fprintf(fp2,"\n\t MUL R1,R0");
fprintf(fp2,"\n\t STORE R0,%s",result);
break;
case '+': fscanf(fp1,"%s %s %s",operand1,operand2,result);
fprintf(fp2,"\n\t LOAD %s,R0",operand1);
fprintf(fp2,"\n\t LOAD %s,R1",operand2);
fprintf(fp2,"\n\t ADD R1,R0");

```



```

fprintf(fp2, "\n \t STORE R0,%s",result);
break;
case '-': fscanf(fp1,"%s %s %s",operand1,operand2,result);
fprintf(fp2, "\n \t LOAD %s,R0",operand1);
fprintf(fp2, "\n \t LOAD %s,R1",operand2);
fprintf(fp2, "\n \t SUB R1,R0");
fprintf(fp2, "\n \t STORE R0,%s",result);
break;
case '/': fscanf(fp1,"%s %s %s",operand1,operand2,result);
fprintf(fp2, "\n \t LOAD %s,R0",operand1);
fprintf(fp2, "\n \t LOAD %s,R1",operand2);
fprintf(fp2, "\n \t DIV R1,R0");
fprintf(fp2, "\n \t STORE R0,%s",result);
break;
case '%': fscanf(fp1,"%s %s %s",operand1,operand2,result);
fprintf(fp2, "\n \t LOAD %s,R0",operand1);
fprintf(fp2, "\n \t LOAD %s,R1",operand2);
fprintf(fp2, "\n \t DIV R1,R0");
fprintf(fp2, "\n \t STORE R0,%s",result);
break;
case '=': fscanf(fp1,"%s %s",operand1,result);
fprintf(fp2, "\n \t STORE %s %s",operand1,result);
break;
case '>': j++;
fscanf(fp1,"%s %s %s",operand1,operand2,result);

```

```
fprintf(fp2, "\n \t LOAD %s,R0",operand1);  
fprintf(fp2, "\n\t JGT %s,label#%s",operand2,result);  
label[no++]=atoi(result);  
break;  
case '<': fscanf(fp1, "%s %s %s",operand1,operand2,result);  
fprintf(fp2, "\n \t LOAD %s,R0",operand1);  
fprintf(fp2, "\n\t JLT %s,label#%d",operand2,result);  
label[no++]=atoi(result);  
break;  
}  
}  
fclose(fp2);  
fclose(fp1);  
fp2=fopen("target.txt", "r");  
if(fp2==NULL)  
{  
printf("Error opening the file\n");  
exit(0);  
}  
do  
{  
ch=fgetc(fp2);  
printf("%c",ch);  
} while(ch!=EOF);  
fclose(fp1);
```

```
return 0;
}
int check_label(int k)
{
    int i;
    for(i=0;i<no;i++)
    {
        if(k==label[i])
            return 1;
    }
    return 0;
}
```

Input:

vi int.txt

=t1 2

[]=a 0 1

[]=a 1 2

[]=a 2 3

*t1 6 t2

+a[2] t2 t3

-a[2] t1 t2

/t3 t2 t2

uminus t2 t2

print t2

```
goto t2 t3
=t3 99
uminus 25 t2
*t2 t3 t3
uminus t1 t1
+t1 t3 t4
```

```
print t4
```

Output:

```
[prasuna@linuxserver ~]$ cc cg.c
```

```
[prasuna@linuxserver ~]$ ./a.out
```

Enter filename of the intermediate code: int.txt

```
STORE t1,2
STORE a[0],1
STORE a[1],2
STORE a[2],3
LOAD t1,R0
LOAD 6,R1
ADD R1,R0
STORE R0,t3
LOAD a[2],R0
LOAD t2,R1
ADD R1,R0
STORE R0,t3
LOAD a[t2],R0
LOAD t1,R1
```

```
SUB R1,R0
STORE R0,t2
LOAD t3,R0
LOAD t2,R1
DIV R1,R0
STORE R0,t2
LOAD t2,R1
STORE R1,t2
LOAD t2,R0
JGT 5,label#11
Label#11: OUT t2
JMP t2,label#13
Label#13: STORE t3,99
LOAD 25,R1
STORE R1,t2
LOAD t2,R0
LOAD t3,R1
MUL R1,R0
STORE R0,t3
LOAD t1,R1
STORE R1,t1
LOAD t1,R0
LOAD t3,R1
ADD R1,R0
STORE R0,t4
```

OUT t4

8. Program to implement Demonstration of constant propagation and common sub expression elimination.

Code Optimization:

If the compiler is an *optimizing compiler*, one of these stages "optimizes" the machine language code so that it either takes less time to run or occupies less memory or sometimes both. Of course, whatever optimizations the compiler does, it must not affect the logic of the program i.e. the optimization must preserve the meaning of the program. One might wonder what type of optimizations the compiler uses to produce efficient machine code? Since in no case the meaning of the program being compiled should be changed, the compiler must inspect the program very thoroughly and find out the suitable optimizations that can be applied. As we may wonder, such a thorough analysis of the program and then finding and applying suitable optimizations is a complex and time consuming process.

A few type of code optimizations that the GNU C Compiler uses so that we can understand how the code is optimized and appreciate the complexity of the optimization process. The GNU C Compiler is a sophisticated optimizing C compiler that uses a large array of optimization techniques to produce efficient code. It may not be possible to describe all of them, so I have chosen some that are interesting and also easy to understand.

- [Constant Folding](#)
- [Common Subexpression Elimination](#)
- [Dead Code Elimination](#)
- [Strength Reduction using Induction Variable](#)
- The GNU C Compiler is a sophisticated optimizing C compiler that uses a large array of optimization techniques to produce efficient code.
- The GNU C Compiler (called GCC henceforth) reads a C program source file, however, it can also produce assembly language source code for the program instead of the object code, so that we can read it and understand how the assembly language program looks.
- We will be generating such assembly language files to see the optimizations being used by the compiler, so it would be beneficial if we see how the assembly language code for a simple C program looks like.

This is optional phase described to improve the intermediate code so that the output runs faster and takes less space. Its output is another intermediate code program that does the some job as the original, but in a way that saves time and / or spaces.

1, Local Optimization:-

There are local transformations that can be applied to a program to make an improvement. For example,

If **A > B** goto **L2**

Goto **L3**

L2 :

This can be replaced by a single statement

If **A < B** goto **L3**

Another important local optimization is the elimination of common sub-expressions

A := B + C + D

E := B + C + F

Might be evaluated as

T1 := B + C

A := T1 + D

E := T1 + F

Take this advantage of the common sub-expressions **B + C**.

2, Loop Optimization:-

Another important source of optimization concerns about **increasing the speed of loops**. A typical loop improvement is to move a computation that produces the same result each time around the loop to a point, in the program just before the loop is entered.

In this we use **gcc** compiler for compiling the program. Why because in **cc** compiler it is not possible to create assembly code file. To create assembly file we use **-S** option , through this we generate assembly file named as filename.s. To optimize this assembly code we use **-O2** option.

Aim: Showing the Code Optimization of different optimization techniques

```
/* This first file simply demonstrates how the assembly program that the
   compiler produces looks like and some peculiarities of the GNU
   assembler that follows some different conventions from MASM/TASM.
```

```
*/
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    printf("Hello, World\n");
```

```
    return 0;
```

```
}
```

OUTPUT:

```
[gnanaprasuna @linuxserver ~]$ gcc -c -S test1.c
```

```
[gnanaprasuna @linuxserver ~]$ gcc -c -S -O2 test1.c
```

```
[gnanaprasuna @linuxserver ~]$ vi test1.s
```

```
.file "test1.c"
```

```
.section .rodata
```

```
.LC0:
```

```
.string "Hello, World"
```

```
.text
```

```
.globl main
```

```
.type main, @function
```



```
main:
    leal    4(%esp), %ecx
    andl    $-16, %esp
    pushl   -4(%ecx)
    pushl   %ebp
    movl    %esp, %ebp
    pushl   %ecx
    subl    $4, %esp
    movl    $.LC0, (%esp)
    call    puts
    movl    $0, %eax
    addl    $4, %esp
    popl    %ecx
    popl    %ebp
    leal    -4(%ecx), %esp
    ret

.size     main, .-main

.ident    "GCC: (GNU) 4.1.2 20080704 (Red Hat 4.1.2-44)"

.section   .note.GNU-stack,"",@progbits
```

Commands for checking the optimization of code:

1.Create a file vi test1.c

2.To generate the assembly language code, create a file test1.c as shown below and give the following command:

- `$ gcc -c -S test1.c`

3.To enable optimization use the following command.

- `gcc -c -S -O2 test2.c`

```
/*test2.c */
```

```
/* Demonstration of constant propagation */
```

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int x, y, z;
```

```
    x = 10;
```

```
    y = x + 45;
```

```
    z = y + 4;
```

```
    printf("The value of z = %d", z);
```

```
    return 0;
```

```
}
```

```
/* end of test2.c */
```

OUTPUT:

```
[gnanaprasuna @linuxserver ~]$ gcc -c -S test2.c
```

```
[gnanaprasuna @linuxserver ~]$ gcc -c -S -O2 test2.c
```

```
[gnanaprasuna @linuxserver ~]$ vi test2.s
```

```
.file "test2.c"
```

```
.section .rodata
```

```
.LC0:
```

```
.string "The value of z = %d"
```

```
.text
```

```
.globl main
```

```
.type main, @function
```

```
main:
```

```
leal 4(%esp), %ecx
```

```
andl $-16, %esp
```

```
pushl -4(%ecx)
```

```
pushl %ebp
```

```
movl %esp, %ebp
```

```
pushl %ecx
```

```
subl $36, %esp
```

```
movl $10, -16(%ebp)
```

```
movl -16(%ebp), %eax
```

```
addl $45, %eax
```

```
movl  %eax, -12(%ebp)
movl  -12(%ebp), %eax
addl  $4, %eax
movl  %eax, -8(%ebp)
movl  -8(%ebp), %eax
movl  %eax, 4(%esp)
movl  $.LC0, (%esp)
call  printf
movl  $0, %eax
addl  $36, %esp
popl  %ecx
popl  %ebp
leal  -4(%ecx), %esp
ret
.size  main, .-main
.ident "GCC: (GNU) 4.1.2 20080704 (Red Hat 4.1.2-44)"
.section .note.GNU-stack,"",@progbits
```

```
/* test3.c */

/* common subexpression elimination, and also of constant propagation */

#include <stdio.h>

int main()
{
    int a, b;

    int x, y, z;

    scanf("%d %d", &a, &b);

    x = a * b;

    if(b >= 4)
    {
        y = a * b;

        z = 0;
    }
    else
    {
        z = a * b * 4;

        y = 0;
    }
}
```

```

printf("x = %d, y = %d, z = %d\n", x, y, z);

return 0;

}

```

OUTPUT:

```
[gnanaprasuna @linuxserver ~]$ gcc -c -S test3.c
```

```
[gnanaprasuna @linuxserver ~]$ gcc -c -S -O2 test3.c
```

```
[gnanaprasuna @linuxserver ~]$ vi test3.s
```

```

.file "test3.c"

.section .rodata.str1.1,"aMS",@progbits,1

```

.LC0:

```
.string "%d %d"
```

.LC1:

```
.string "x = %d, y = %d, z = %d\n"
```

```
.text
```

```
.p2align 4,,15
```

.globl main

```
.type main, @function
```

main:

```
leal 4(%esp), %ecx
```

```
andl $-16, %esp
```

```
pushl -4(%ecx)
```

```
pushl %ebp
```

```
movl %esp, %ebp
```

```
pushl %ebx
```

```
xorl  %ebx, %ebx
pushl %ecx
subl  $32, %esp
leal  -16(%ebp), %eax
movl  %eax, 8(%esp)
leal  -12(%ebp), %eax
movl  %eax, 4(%esp)
movl  $.LC0, (%esp)
call  scanf
movl  -16(%ebp), %eax
movl  %eax, %edx
imull -12(%ebp), %edx
cmpl  $3, %eax
movl  %edx, %ecx
jg    .L4
leal  0(%edx,4), %ebx
xorl  %ecx, %ecx
```

.L4:

```
movl  %ebx, 12(%esp)
movl  %ecx, 8(%esp)
movl  %edx, 4(%esp)
movl  $.LC1, (%esp)
call  printf
addl  $32, %esp
xorl  %eax, %eax
```

```
popl    %ecx
popl    %ebx
popl    %ebp
leal    -4(%ecx), %esp
ret

.size   main, .-main
.ident  "GCC: (GNU) 4.1.2 20080704 (Red Hat 4.1.2-44)"
.section .note.GNU-stack,"",@progbits
```



```
/* test4.c */

/* demonstration of dead code elimination */

#include <stdio.h>

int main()
{
    int x;

    scanf("%d", &x);

    if(x < 0 && x > 0)
    {
        x = 99;
        printf("Hello. Inside the if!!!");
    }

    return 0;
}

/* end of test4.c */

/* ----- */
```

OUTPUT:

```
[gnanaprasuna@linuxserver ~]$ gcc -c -S test4.c
```

```
[gnanaprasuna @linuxserver ~]$ gcc -c -S -O2 test4.c
```

```
[gnanaprasuna @linuxserver ~]$ vi test4.s
```

```
.section .rodata.str1.1,"aMS",@progbits,1
```

```
.LC0:
```

```
.string "%d"
```

```
.text
```

```
.p2align 4,,15
```

```
.globl main
```

```
.type main, @function
```

```
main:
```

```
leal 4(%esp), %ecx
```

```
andl $-16, %esp
```

```
pushl -4(%ecx)
```

```
pushl %ebp
```

```
movl %esp, %ebp
```

```
pushl %ecx
```

```
subl $36, %esp
```

```
leal  -8(%ebp), %eax
movl  %eax, 4(%esp)
movl  $.LC0, (%esp)
call  scanf
addl  $36, %esp
xorl  %eax, %eax
popl  %ecx
popl  %ebp
leal  -4(%ecx), %esp
ret

.size  main, .-main

.ident  "GCC: (GNU) 4.1.2 20080704 (Red Hat 4.1.2-44)"

.section   .note.GNU-stack,"",@progbits
```